

Advantages and Disadvantages of Non-Generic Collection in C#

In this article, I am going to discuss the **Advantages and Disadvantages of the Non-Generic Collection in C#**. Please read our previous article where we discussed the [Non-Generic SortedList Collection Class in C#](#) with Examples. Here we will discuss the advantages and disadvantages of using the ArrayList collection class which can also be applied to other non-generic collection classes such as Stack, Queue, Hashtable, SortedList, etc.

Advantages of using Non-Generic Collection Classes in C#:

As we already discussed the Non-Generic Collection classes can grow in size automatically when we add items to the collection and this is the advantage. Let us prove this with an example.

In the following example, we create a collection i.e. Numbers of the type ArrayList with the initial size 3. But we then add 4 elements into the collection, and we did not get any errors, no compile time error, and no run-time error and it works as expected. Hence, it proves that collections like [ArrayList](#), [Stack](#), [Queue](#), [Hashtable](#), [SortedList](#), etc. can grow in size dynamically when we add items to the collection. If this is an integer array, when we add the fourth element to the collection then we will get the index out of the bound run time exception.

```
using System;
using System.Collections;
namespace CollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ArrayList Numbers = new ArrayList(3);
            Numbers.Add(100);
            Numbers.Add(200);
            Numbers.Add(300);
            Numbers.Add(400);
            foreach (int Number in Numbers)
            {
                Console.WriteLine(Number + " ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
100 200 300 400
```

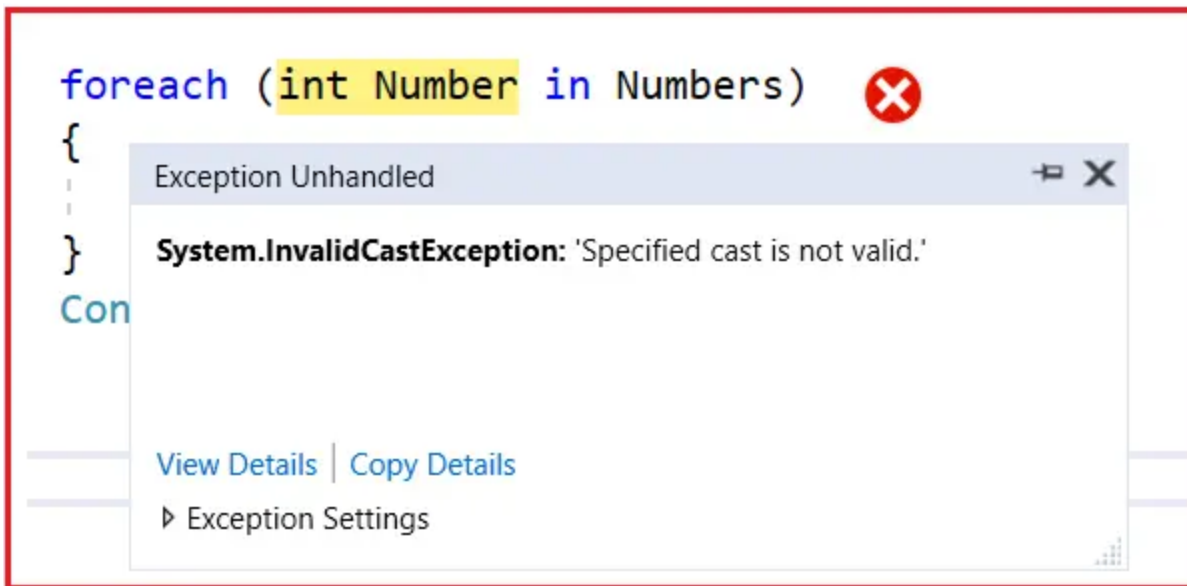
The non-generic collection classes such as **ArrayList**, **Stack**, **Queue**, **SortedList**, and **Hashtable**, etc. provide several useful methods to add and remove items to the collection as well as they also provide some methods using which we can add or remove items from the middle of a collection and this is the other benefits which we get in non-generic collection classes in C# which we cannot get in arrays. Now, let us proceed and try to understand the disadvantages of Non-Generic Collection Classes in C#.

Disadvantages of using Non-Generic Collection Classes in C#:

The Non-Generic Collection Classes such as **ArrayList**, **Stack**, **Queue**, **Hashtable**, **SortedList**, etc operate on the object data type. As they operate on object data type hence they are loosely typed. Loosely typed means you can store any type of value in the collection. Because of this loosely typed nature, we may get runtime errors. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace CollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ArrayList Numbers = new ArrayList(3);
            Numbers.Add(100);
            Numbers.Add(200);
            Numbers.Add(300);
            //It is also possible to store string values
            Numbers.Add("Hi");
            foreach (int Number in Numbers)
            {
                Console.Write(Number + " ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:



Not only do we get run time errors because of the loosely-typed nature, but it also affects the performance of the application due to boxing and unboxing. Let us understand this with an example.

In the following example, we create a Non-Generic Collection i.e. Numbers of the type ArrayList with the initial size 2. Then we are storing two elements like 100 and 200. These two items 100 and 200 are integers as well as value types.

The Collection Classes belong to System.Collections namespace operates on the object data type. The object data type in C# is a reference data type. So the value that we store in the collection is converted to reference type. So in our example, the values 100 and 200 are boxed and converted into the reference type. In our example, we just stored two values. Consider a scenario where we need to store 1000 integer values. Then all 1000 integers need to be boxed, meaning they are converted into reference types and then stored in the collection.

Similarly, when we want to retrieve the items from the collection, then again we need to convert the object type back to the integer type meaning performing an unboxing. So this unnecessary boxing and unboxing happen behind the scenes every time we add and retrieve value types to the collection. So if you are operating on a large collection of value types then it may degrade the performance of your application. So, always try to avoid boxing and unboxing while developing your real-time application.

```
using System;
using System.Collections;
namespace CollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ArrayList Numbers = new ArrayList(2);
            // Boxing happens - Converting Value type (100, 200) to reference
            // Means Integer to object type
            Numbers.Add(100);
            Numbers.Add(200);
            // Unboxing happens - 100 and 200 stored as object type
            // now converted to Integer type
        }
    }
}
```

```

        foreach (int Number in Numbers)
        {
            Console.WriteLine(Number + " ");
        }
        Console.ReadKey();
    }
}

```

Problems with Non-Generic Collection Classes in C#:

So, in short, we can say that the Non-Generic Collection Classes in C# are not type-safe as they operate on object data types so they can store any type of value.

1. Array is type-safe
2. Array List, HashTable, Stack, SortedList, and Queue are not type-safe

For example, if I want to store n no of integer values

1. I cannot go with an array as arrays are fixed in length. In this case, the length is unknown
2. I can go with an ArrayList or HashTable but if we go with an ArrayList or HashTable then there is a chance of storing other types of values as they are not type-safe as they operate on the object data type

So the solution is Generic Collections in C#.

1. **Array**: Type-Safe but Fixed Length
2. **Collections**: Auto Resizing but not Type-Safe
3. **Generic Collections**: Type-Safe and Auto-Resizing

In the next article, I am going to discuss the **Generic Collections in C#**. Here, in this article, I try to explain the **Advantages and Disadvantages of Collection Classes in C#** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about the Advantages of the Non-Generic Collections in C# article.